

Analyses bivariées et prédiction opérationnelle

Le but de cette note est d'expliquer comment, à partir de la table [resultat.etp_ope_sec_delais_1sll](#), nous avons associé des informations relatives à la température aux interventions de la Ville de Lyon du SDMIS.

À partir de cette jointure et d'autres variables, nous avons mené diverses analyses bivariées suivies de la création d'un modèle prédictif basé sur la température, la saisonnalité, les périodes de vacances scolaires, le jour de la semaine et des créneaux horaires.

Les résultats du modèle prédictif sont analysés à travers la création de graphiques interactifs. Leur visualisation est possible grâce à une source de publication fournie par GitHub nommée "Pages" qui crée un lien. Les graphiques sont ainsi publiés comme une page web

Méthodologie qui utilise 1 logiciel et plusieurs langages de programmation :

- *FME* : Appel à l'API <https://open-meteo.com/> et affiliation à chaque intervention d'une réponse en JSON de la base de données météorologique
- *Python* : langage de programmation majoritairement utilisé pour le développement d'application web, le développement de logiciels, l'automatisation de processus et l'exploration des données. Ce langage a été utilisé pour formater la réponse API et encoder les données, pour les analyses bivariées et pour la création d'un JSON sur lequel le graphique interactif s'appuie.
- *HTML* : langage de balisage utilisé pour structurer et organiser le contenu d'une page web. Il nous a permis de structurer la page web accueillant le graphique interactif (plusieurs pages, organisation des widgets, placements des divers éléments, titres, etc.)
- *JavaScript* : langage de programmation permettant de définir l'interactivité d'une page web. Il nous a permis de définir quelles étaient les actions déclenchées à chaque clic sur le graphique interactif.
- *CSS* : langage informatique utilisé pour mettre en forme la page web (stylisation, polices, etc.). Il nous a permis de styliser le graphique interactif (couleurs, tailles des polices, espacements, marges, etc.)

Structure globale de la méthodologie :

[Organisation générale de la méthodologie du modèle prédictif.canvas](#)

Première partie de la méthodologie :

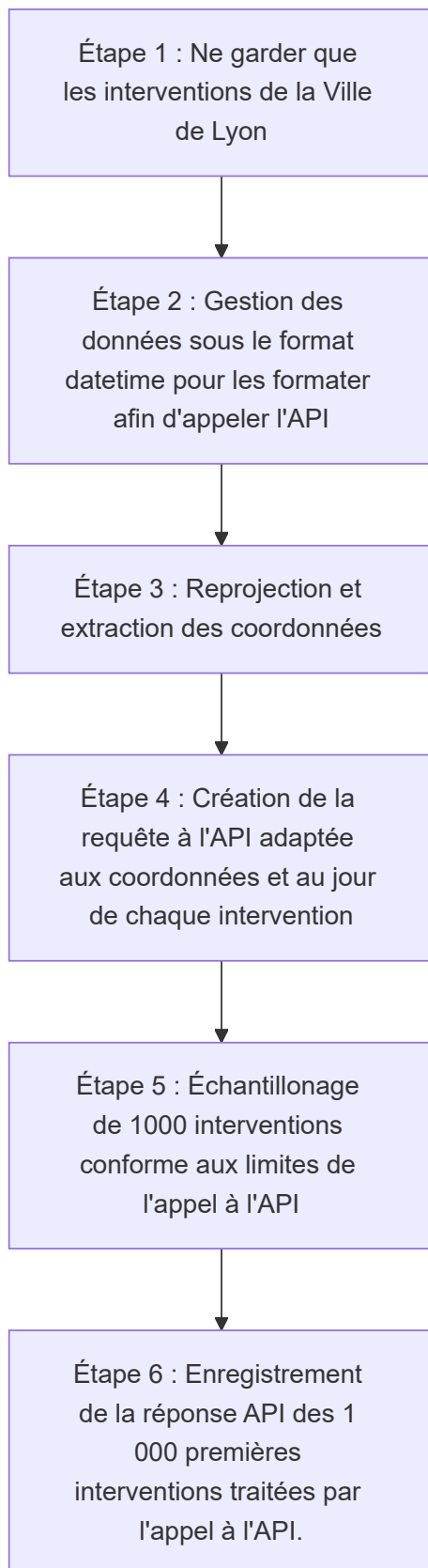
FME

La création d'un script FME à partir du périmètre de la Ville de Lyon et des interventions de la table [resultat.etp_ope_sec_delais_1sll](#).

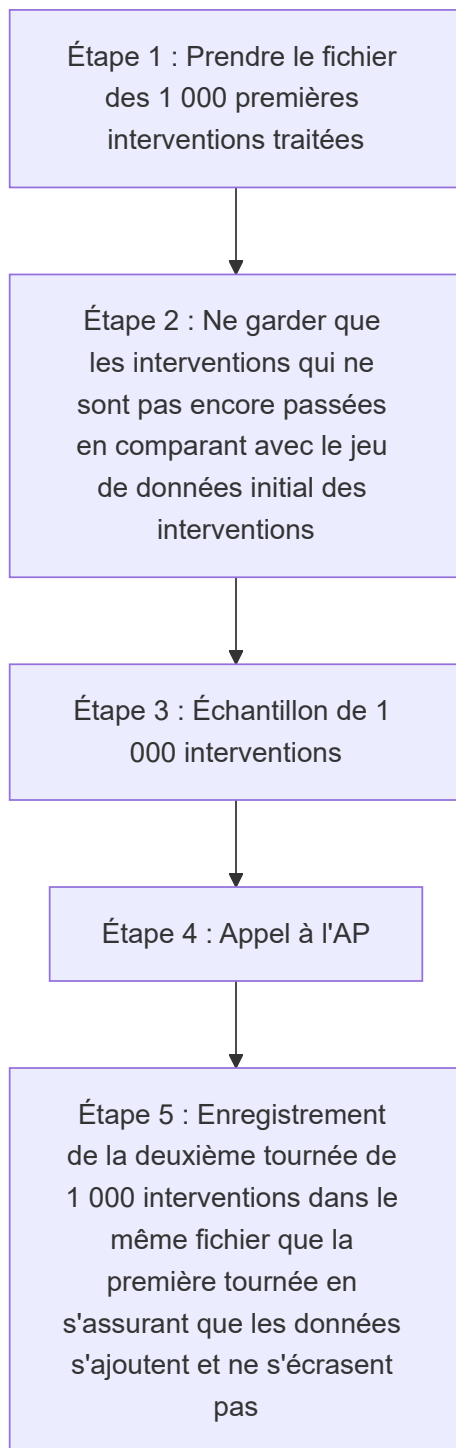
Lien vers le script FME, **dans lequel chaque transformer est commenté** :

"\\filer2.sdmis.fr\\SIG\\Bac_a_sable\\Charlotte\\FME\\Jointure_Temp_Inter.fmw"

Le script suit la logique suivante :



À cette étape, les 1000 premières interventions sont enregistrées dans un .gpkg avec une nouvelle colonne nommée `_reponse_api` qui nous indique des données météorologiques horaires de la journée de l'intervention (en JSON). En effet, nous n'avons pas pu requêter l'API directement sur l'heure de l'intervention, nous récupérons ainsi toutes les données météorologiques de la journée (24 données pour 24 heures) et nous récupérerons la donnée horaire de l'heure de l'intervention grâce à Python. Pour les interventions suivantes, nous faisons une boucle sur l'API en prenant soin de ne faire repasser dans la boucle que les interventions n'y étant pas encore passées :



Le fichier final contient ainsi pour chaque intervention de la Ville de Lyon du 01/01/2019 au 24/06/2026 (date de la dernière boucle effectuée, $n=245\,305$ interventions) une colonne `_reponse_api` contenant des informations météorologiques horaires, le but est de récupérer ces informations pour l'heure de chaque intervention.

Le script FME et le fichier "inter_meteo_reussi" peuvent être dans un même dossier mais séparés du dossier contenant les données issues du modèle prédictif, le code html, le code css et le code JavaScript qui seront déposés sur GitHub.

L'idée est que le traitement des données brutes est public mais pas les données en elles-mêmes.

2ème partie de la méthodologie :

PASSAGE SUR PYTHON

Objectif : extraire la réponse de l'API et mener des analyses bivariées et prédictives

```

#Ajout des librairies nécessaires au nettoyage/formatage des données et à leurs analyses
et visualisations
import geopandas as gpd
import pandas as pd
import numpy as np
import json
import os
import plotly.graph_objects as go
import matplotlib.pyplot as plt
from vacances_scolaires_france import SchoolHolidayDates
from scipy import stats
from sklearn.ensemble import RandomForestRegressor
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from pathlib import Path

#Extraction des données météorologiques
#Import du jeu de données des interventions et de leur réponse API
gdf = gpd.read_file('Chemin vers le fichier .gpkg des interventions après la boucle sur
l'API = le writer final du script FME )

#Conversion des données temporelles en datetime et harmonisation des fuseaux horaires
#On transforme la colonne dh_debut_inter en datetime dans une nouvelle colonne
'datetime_local' pour s'assurer du format, même si cette colonne était déjà en datetime
gdf['datetime_local'] = pd.to_datetime(gdf['dh_debut_inter'])

#Ajout de l'information hiver (CET)/ été (CEST) à la colonne datetime_local
gdf['datetime_local'] = gdf['datetime_local'].dt.tz_localize(
    'Europe/Paris',
    ambiguous='NaT',
    nonexistent='shift_forward'
)
#Résultat une colonne datetime_local avec le format AAAA-MM-JJ HH:MM:SS+HH:SS (le +
représentant le nombre d'heure de décalage entre heure locale et heure UTC)

#Suppression du détail des minutes et des secondes pour permettre d'associer les
informations météorologiques avec l'heure (on arrondi à l'heure en mettant 0 pour les
minutes et les secondes)
gdf['datetime_local'] = gdf['datetime_local'].dt.floor('h')

#Conversion de datetime_local en heure UTC dans une nouvelle colonne datetime_utc et
visualisation du résultat
#concrètement, les + 1 et les + 2 se retirent pour avoir l'heure en UTC (qui représente
-1 ou -2 heure par rapport à l'heure locale)

gdf['datetime_utc'] = gdf['datetime_local'].dt.tz_convert('UTC')
print(gdf[['dh_debut_inter', 'datetime_local', 'datetime_utc']].head(10))

#Modification de l'affichage de l'heure de la colonne datetime_utc dans une nouvelle
colonne heure_utc_cible pour qu'il soit dans le même format que l'heure dans la réponse
de l'API (format AAAA-MM-JJTHH:MM alors que dans la colonne datetime_utc le format était
AAAA-MM-JJ HH:MM:SS+HH:MM)
gdf['heure_utc_cible'] = gdf['datetime_utc'].dt.strftime('%Y-%m-%dT%H:00')

```

```

#Extraction des données météorologiques grâce à l'harmonisation des heures et de leurs
formats
def extraire_meteo(row):
    try:
        data = json.loads(row['_response_api'])
        times = data['hourly']['time']
        cible = row['heure_utc_cible']

        if cible not in times:
            return pd.Series({
                'temperature_2m': None,
                'windspeed_10m': None,
                'meteo_status': 'heure_hors_plage'
            })

        idx = times.index(cible)
        return pd.Series({
            'temperature_2m': data['hourly']['temperature_2m'][idx],
            'windspeed_10m': data['hourly']['windspeed_10m'][idx],
            'meteo_status': 'ok'
        }) #Possible d'ajouter d'autres variables météo ici (précipitations, humidité,
        etc.)

    except (TypeError, json.JSONDecodeError, KeyError, ValueError, AttributeError):

        return pd.Series({
            'temperature_2m': None,
            'windspeed_10m': None,
            'meteo_status': 'erreur_parsing'
        })

meteo_cols = gdf.apply(extraire_meteo, axis=1)
gdf = pd.concat([gdf, meteo_cols], axis=1)

#Visualisation du résultat sur 10 interventions
print(gdf[['heure_utc_cible', 'temperature_2m',
            'windspeed_10m', 'meteo_status']].head(20))

#Analyse des interventions n'ayant pas de données météorologiques
#Le problème survient pour les interventions en heure d'été (CEST, UTC+2) entre 0 h et 1
h du matin (heure locale) : elles se convertissent en 22h ou 23h UTC la veille, qui n'est
pas dans le tableau du jour. Exemple concret : une intervention le 15 juillet à 00 h 30
locale → 22 h 30 UTC le 14 juillet → heure UTC cible 2022-07-14T22:00, absente du JSON
qui commence à 2022-07-15T00:00

#Suppression des interventions 'hors-plage'
gdf = gdf[gdf["meteo_status"] == "ok"].reset_index(drop=True)
print(f"Nombre d'interventions après suppression des hors plage' : {len(gdf)}")
# Nombre d'interventions supprimées : 12 155

```

Ayant obtenu une température associée à chaque intervention, l'analyse de la corrélation entre le nombre d'interventions par créneau et la moyenne des températures par créneau est possible mais il est nécessaire d'agréger les données par jour et par créneau :

```
#####
#Analyse de la corrélation
#Création de champ
#Date (va récupérer la date dans le champ datetime_local)
gdf["date"] = gdf["datetime_local"].dt.date

#Création des créneaux en se basant sur la colonne heure ("Nuit (0-6h)", "Matin (6-12h)",
"Après-midi (12-18h)", "Soir (18-24h)")
gdf["creneau"] = pd.cut(
    gdf["heure"].astype(int),
    bins=[0, 6, 12, 18, 24],
    labels=["Nuit (0-6h)", "Matin (6-12h)", "Après-midi (12-18h)", "Soir (18-24h)"],
    right=False
)

#####
#Analyse de la corrélation entre le nombre d'interventions par jour et par créneau et les
moyennes de températures par créneau
#Paramétrage de la figure
plt.rcParams.update({
    "font.size": 11,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.alpha": 0.25,
    "figure.facecolor": "white",

})
police= "Arial"
taille_police = 12
taille_titre_axe = taille_police + 1
largeur_fig = 700
hauteur_fig= 500

#Agrégation du nombre d'interventions par jour et par créneau et moyennes des
températures par jour et par créneau
df_agg = (
    gdf
    .groupby(["date", "creneau"], observed=True)
    .agg(
        nb_interventions = ("id_intervention_sdis", "count"),
        temp_moy          = ("temperature_2m", "mean"),
    )
    .reset_index()
)

#NB : on peut imaginer la corrélation avec d'autres variables météorologiques telles que
les précipitations ou la vitesse de vent. À la place de temp_moy on peut insérer une
vitesse moyenne de vent ou des précipitations moyennes par créneau

#Afficher le nombre de créneaux résultants de cette agrégation :
print(f"Nombre de créneaux étudiés : {len(df_agg)}")

#On détermine les seuils de grand chaud et de grand froid selon la distribution
statistique des moyennes de températures par créneau (5ème et 95ème percentile)
seuil_canicule_p95 = df_agg["temp_moy"].quantile(0.95)
seuil_grand_froid_p05 = df_agg["temp_moy"].quantile(0.05)
```

```

#On calcule la corrélation et on la visualise
bins = pd.cut(df_agg["temp_moy"], bins=20)
stats_bin = (
    df_agg
    .groupby(bins, observed=True)["nb_interventions"]
    .mean()
)

centres = stats_bin.index.map(lambda i: i.mid).astype(float)
pente, ordonnee, r_value, p_val, err = stats.linregress(
    df_agg["temp_moy"],
    df_agg["nb_interventions"]
)

r2 = r_value**2
x = np.linspace(
    df_agg["temp_moy"].min(),
    df_agg["temp_moy"].max(),
    200
)

fig1 = go.Figure()
fig1.add_trace(go.Scatter(
    x=df_agg["temp_moy"],
    y=df_agg["nb_interventions"],
    mode="markers",
    name="Nombre d'interventions par créneau",
    marker=dict(size=5, color="#000000", opacity=0.03)
))

fig1.add_trace(go.Scatter(
    x=centres,
    y=stats_bin.values,
    mode="lines+markers",
    name="Moyenne d'intervention selon la température",
    line=dict(color="rgba(35, 139, 69, 0.85)", width=2),
    marker=dict(size=6)
))

fig1.add_trace(go.Scatter(
    x=x,
    y=ordonnee + pente * x,
    mode="lines",
    name="Tendance linéaire",
    line=dict(color="#000000", dash="dash", width=1.5)
))

fig1.add_vline(x=SEUIL_GRAND_FROID_P05_ALL, line=dict(color="#4292c6", dash="dot"))
fig1.add_vline(x=SEUIL_CANICULE_P95_ALL, line=dict(color="#cb181d", dash="dot"))

fig1.add_annotation(
    x=SEUIL_GRAND_FROID_P05_ALL - 1, y=1, yref="paper",
    text="Grand froid", showarrow=False, textangle=-90,
    font=dict(family=police, color="#4292c6", size=taille_police)
)

```

```

)

fig1.add_annotation(
    x=SEUIL_CANICULE_P95_ALL - 1, y=1, yref="paper",
    text="Grande chaleur", showarrow=False, textangle=-90,
    font=dict(family=police, color="#cb181d", size=taille_police)
)

fig1.update_layout(
    template="plotly_white",
    font=dict(family=police, size=taille_police, color="black"),
    xaxis_title="Température moyenne par créneau (°C)",
    yaxis_title="Nombre d'interventions par créneau",
    hovermode="closest",
    margin=dict(l=60, r=30, t=60, b=60),
    width=largeur_fig,
    height=hauteur_fig,
    legend=dict(
        x=0.65, y=0.80, xanchor="right", yanchor="bottom",
        bgcolor="rgba(255,255,255,0.0)",
        bordercolor="lightgray", borderwidth=0.5,
        font=dict(family=police, size=taille_police)
    )
)

fig1.update_xaxes(
    range=[df_agg["temp_moy"].min() - 1, df_agg["temp_moy"].max() + 1],
    title_font=dict(family=police, size=taille_titre_axe),
    tickfont=dict(family=police, size=taille_police)
)

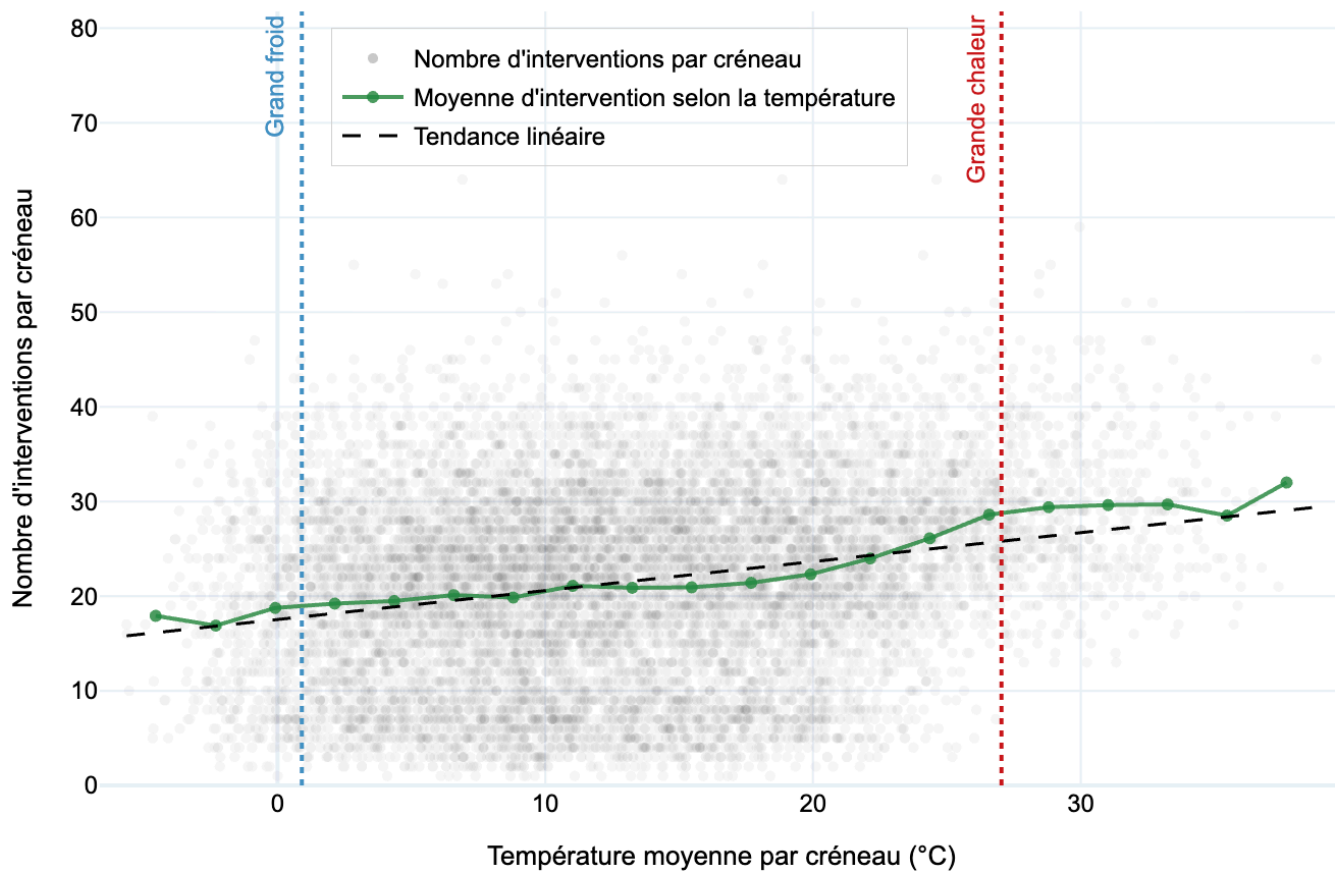
fig1.update_yaxes(
    rangemode="tozero",
    title_font=dict(family=police, size=taille_titre_axe),
    tickfont=dict(family=police, size=taille_police)
)

fig1.show()

#Affiche l'équation de la droite, sa significativité et le coefficient de détermination
print(f"Équation : nb_interventions = {pente:.3f} × temp_moy + {ordonnee:.3f}")
print(f"R² = {r2:.3f}")
print(f"p-value = {p_val:.3e}")

```

À l'issue de cette première analyse, la figure suivante nous permet de visualiser la corrélation entre le nombre d'interventions par créneau et les températures moyennes par créneau :



Dans le but de comprendre l'influence d'autres variables sur le nombre d'interventions par créneau/jour, nous avons mené une vague d'analyses bivariées type ANOVA à un facteur étant donné que ce sont des variables qualitatives (jour de la semaine, saison, période de vacances OUI/NON, créneau). Ces variables permettent de répondre (par exemple) à la question : est-ce que la saisonnalité ou le fait d'être en vacances influencent le nombre d'interventions par jour et par créneau ?

```
#Analyse des ANOVA
#Création des champs saison, vacances et jour de la semaine (déjà présent dans le jeu de
données initial mais on le recrée pour être sûr de sa lisibilité par Python + le jour de
la semaine n'est pas présent dans le jeu de données agrégé)
def determiner_saison(date):
    mois = date.month
    if mois in [12, 1, 2]:
        return "Hiver"
    elif mois in [3, 4, 5]:
        return "Printemps"
    elif mois in [6, 7, 8]:
        return "Ete"
    else:
        return "Automne"

vacances = SchoolHolidayDates()
def est_en_vacances(date):
    try:
        d = date.date() if hasattr(date, "date") else date
        return "Oui" if vacances.is_holiday_for_zone(d, "A") else "Non"
    except:
        return "Non"

#Application des définitions sur le jeu de données agrégé
df_agg["saison"] = df_agg["date"].apply(determiner_saison)
```

```
df_agg["vacances"] = df_agg["date"].apply(est_en_vacances)
```

```
#Ajout du jour de la semaine et conversion du champ date en datetime car il n'était pas en datetime sur le jeu de données agrégé
```

```
df_agg["date"] = pd.to_datetime(df_agg["date"])
```

```
df_agg["jour_semaine"] = df_agg["date"].dt.day_name()
```

RAPPEL : le jeu de données agrégé suit cette structure :

	date	creneau	nb_interventions	temp_moy	saison	vacances	\
0	2019-01-01	Nuit (0-6h)	64	6.906250	Hiver	Oui	
1	2019-01-01	Matin (6-12h)	23	6.217391	Hiver	Oui	
2	2019-01-01	Après-midi (12-18h)	19	7.215789	Hiver	Oui	
3	2019-01-01	Soir (18-24h)	17	5.347059	Hiver	Oui	
4	2019-01-02	Nuit (0-6h)	10	5.280000	Hiver	Oui	
5	2019-01-02	Matin (6-12h)	20	4.240000	Hiver	Oui	
6	2019-01-02	Après-midi (12-18h)	20	4.935000	Hiver	Oui	
7	2019-01-02	Soir (18-24h)	31	1.970968	Hiver	Oui	
8	2019-01-03	Nuit (0-6h)	7	1.114286	Hiver	Oui	
9	2019-01-03	Matin (6-12h)	18	0.055556	Hiver	Oui	
	jour_semaine						
0	Tuesday						
1	Tuesday						
2	Tuesday						
3	Tuesday						
4	Wednesday						
5	Wednesday						
6	Wednesday						
7	Wednesday						
8	Thursday						
9	Thursday						

Ainsi, chaque jour est répété 4 fois pour les 4 créneaux avec à chaque fois une indication sur : la température moyenne, le nombre d'intervention, la saison, le jour de la semaine et le fait d'être en vacances (oui/non).

Ce jeu de données nous permet ainsi de procéder à des ANOVA :

```
#Vérification de l'homogénéité et de l'équilibre des groupes avant de mobiliser le test de Fisher
```

```
print(df_agg["vacances"].value_counts())
```

```
print(df_agg["saison"].value_counts())
```

```
print(df_agg["creneau"].value_counts())
```

```
print(df_agg["jour_semaine"].value_counts())
```

```
#ANOVA
```

```
#Définition des variables à analyser (les variables qualitatives/catégorielles)
```

```
variables_categorielles = [
```

```
    "saison",
```

```
    "vacances",
```

```
    "creneau",
```

```
    "jour_semaine"
```

```
]
```

#Interpréter l'êta carré (mesure la proportion de la variance du nombre d'interventions expliquée par l'une des variables qualitatives)

```
def interpreter_eta2(eta2):
```

```
    if eta2 < 0.01:
        return "négligeable"
    elif eta2 < 0.06:
        return "petit"
    elif eta2 < 0.14:
        return "moyen"
    else:
        return "grand"
```

#Définir l'ANOVA à un facteur

```
def analyse_categorielle(df, var_cat, var_num):
```

```
    groupes = [grp[var_num].dropna().values
                for _, grp in df.groupby(var_cat, observed=True)]

    f_stat, p_value = stats.f_oneway(*groupes)
    grand_mean = df[var_num].mean()
    ss_between = sum(len(g) * (g.mean() - grand_mean) ** 2 for g in groupes)
    ss_total = ((df[var_num] - grand_mean) ** 2).sum()
    eta2 = ss_between / ss_total
```

```
    print(f"--- {var_cat} ---")
    print(f"ANOVA (Fisher) : F = {f_stat:.2f}, p = {p_value:.4g}")
    print(f" $\eta^2$  = {eta2*100:.1f}% → effet {interpreter_eta2(eta2)}")
    print()
```

```
    return f_stat, p_value, eta2
```

#Application de l'ANOVA sur les variables qualitatives/catégorielles

```
resultats = []
```

```
for var_cat in variables_categorielles:
```

```
    f_stat, p_value, eta2 = analyse_categorielle(
        df_agg, var_cat, "nb_interventions"
    )
```

```
    resultats.append({
        "variable": var_cat,
        "F": round(f_stat, 2),
        "p_value": p_value,
        "eta2_pct": round(eta2 * 100, 2),
        "effet": interpreter_eta2(eta2)
    })
```

```
df_resultats = pd.DataFrame(resultats).sort_values(
    "eta2_pct", ascending=False
)
```

```
print(df_resultats)
```

#NB : on peut imaginer mettre plus de variables catégorielles dans l'analyse de l'ANOVA, il suffit de les ajouter à la variable Python "variables_categorielles"

Enfin, le modèle prédictif repose sur un modèle de type Random Forest qui ne nécessite pas forcément une corrélation/anova entre variables prédictives (température, saison, vacances, jour de la semaine, créneau) et variable à prédire (le nombre d'interventions par créneau). Nous avons tout de même mené ces analyses dans le but de comprendre leur influence sur l'activité opérationnelle.

Concrètement, on entraîne un algorithme à deviner, pour chaque créneau de 6h et chaque jour de l'année, combien d'interventions auront lieu en se basant sur ce qui s'est passé les années précédentes (en regardant la saisonnalité, les températures, les périodes de vacances, etc.).

```
#Création du modèle prédictif
#Séparation chronologique (et non aléatoire) : 80% des jours les plus anciens pour
l'entraînement, 20% des jours les plus récents pour le test.
date_limite_global = df_agg["date"].quantile(0.8)
#Les données d'entraînement
train_global = df_agg[df_agg["date"] < date_limite_global].copy()
#Les données de test
test_global = df_agg[df_agg["date"] >= date_limite_global].copy()

#Variables prédictives utilisées par le modèle
var_num = ["temp_moy"]
var_cat = ["saison", "vacances", "creneau", "jour_semaine"]
var_predictives = var_num + var_cat

#Encodage des variables pour qu'elles soient lisibles par le modèle (ex : chaque saison
est séparée dans une colonne est_hiver / est_ete / est_printemps/ estautomne avec des
valeurs 1 = oui, 0 = non)
preprocesseur = ColumnTransformer(transformers=[
    ("num", "passthrough", var_num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), var_cat),
])

#Construction du modèle en lui-même, 400 arbres de décision qui fonctionnent comme des
questions/réponse en cascade (est-qu'on est en vacances ? Si oui, est-ce le soir ? et
prédit un nombre d'intervention en fonction des réponses)
modele_rf = Pipeline(steps=[
    ("prep", preprocesseur),
    ("rf", RandomForestRegressor(
        n_estimators=400,          #nombre d'arbres dans la forêt
        min_samples_leaf=3,       #évite que les arbres n'apprennent "par cœur" des cas
trop rares
        random_state=42,         #figeage du hasard pour pouvoir reproduire exactement le
même résultat
        n_jobs=-1
    )),
])

#Entraînement : le modèle parcourt les données d'entraînement (le passé) et apprend les
liens entre les variables prédictives et le nombre d'interventions observées
modele_rf.fit(train_global[var_predictives], train_global["nb_interventions"])

#Test : on demande au modèle de prédire le nombre d'interventions sur les données test
(des données qu'il n'a jamais vues lors de l'entraînement) puis on compare sa prédiction
avec la réalité pour juger sa qualité
pred_test_rf = modele_rf.predict(test_global[var_predictives])

#Calcul des indicateurs de précision (MAE, RMSE, R²)
```

```

mae_rf = mean_absolute_error(test_global["nb_interventions"], pred_test_rf)
rmse_rf = np.sqrt(mean_squared_error(test_global["nb_interventions"], pred_test_rf))
r2_rf = r2_score(test_global["nb_interventions"], pred_test_rf)

#Calcul de l'intérêt du modèle comparé à si on regarde simplement les moyennes
historiques pour évaluer les futures moyennes
moyenne_naive = train_global.groupby("creneau", observed=True)["nb_interventions"].mean()
pred_naive_test = test_global["creneau"].map(moyenne_naive)
mae_naive = mean_absolute_error(test_global["nb_interventions"], pred_naive_test)
amelioration_pct = (mae_naive - mae_rf) / mae_naive * 100

#Visualisation des indicateurs de performance
print(f"Variables utilisées : {' , '.join(var_predictives)}")
print(f"MAE : {mae_rf:.2f} interventions")
print(f"RMSE : {rmse_rf:.2f}")
print(f"R² : {r2_rf*100:.1f}%")
print(f"MAE modèle : {mae_rf:.2f} | MAE moyenne historique seule : {mae_naive:.2f}")
print(f"Amélioration apportée par le modèle : {amelioration_pct:.1f}%")

#Calcul de l'importance de chaque variable dans la prédiction et visualisation
noms_features = modele_rf.named_steps["prep"].get_feature_names_out()
importances = modele_rf.named_steps["rf"].feature_importances_
importance_par_variable = {}
for nom, imp in zip(noms_features, importances):
    for v in var_predictives:
        if nom.replace("num__", "").replace("cat__", "").startswith(v):
            importance_par_variable[v] = importance_par_variable.get(v, 0) + imp
            break

#Petit graphique pour résumer l'importance des variables prédictives sur le modèle
noms_lisibles = {
    "creneau": "Créneau horaire", "jour_semaine": "Jour de la semaine",
    "temp_moy": "Température moyenne", "vacances": "Période de vacances", "saison":
    "Saison"
}

items = sorted(importance_par_variable.items(), key=lambda x: x[1])
labels = [noms_lisibles.get(k, k) for k, v in items]
valeurs = [v*100 for k, v in items]

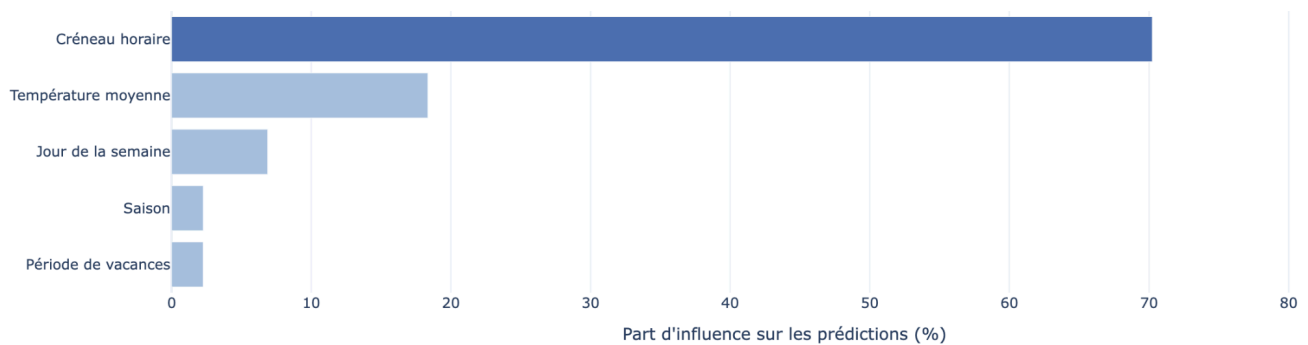
fig_imp = go.Figure(go.Bar(
    x=valeurs, y=labels, orientation="h",
    marker_color=["#4C72B0" if v == max(valeurs) else "#A6C1E0" for v in valeurs],
))

fig_imp.update_layout(
    title="Quelles variables influencent le plus les prédictions du modèle ?",
    xaxis_title="Part d'influence sur les prédictions (%)",
    template="plotly_white", height=350, margin=dict(l=180, t=60, b=40),
    xaxis=dict(range=[0, max(valeurs)*1.25])
)

fig_imp.show()

```

Résultat de la figure résumant l'importance de chaque variable prédictif sur le modèle :
Quelles variables influencent le plus les prédictions du modèle ?



À la suite de la création du modèle prédictif et de son évaluation, plusieurs scénarios de simulation de réchauffement climatique peuvent être mobilisés pour évaluer l'impact du réchauffement climatique sur la volumétrie opérationnelle (2026). Ainsi, nous avons créé une "grille" avec en ligne le couple jour/créneau (donc 4 lignes par jour) pour l'année 2026 en y indiquant la saison, le fait d'être en vacances ou non, le jour de la semaine et une température de base suivant la normale de saison en se basant sur les données historiques des températures.

```
#On limite le jeu de données à 2019-2025 pour calculer les moyennes historiques sans
prendre en compte les données de janvier à juin 2026
df_hist = df_agg[df_agg["date"].dt.year <= 2025].copy()

annee_cible = 2026 #Changer la date si l'analyse se fait sur des années différentes (par
exemple en refaisant la méthodologie sur les données de toute l'année 2026 pour prédire
2027, il faudrait cependant refaire le script FME sur les interventions manquantes de
2026)

#Création d'une grille contenant tous les jours de l'année à prédire (du 01/01/2026 au
31/12/2026)
dates_annee = pd.date_range(f"{annee_cible}-01-01", f"{annee_cible}-12-31", freq="D")
creneaux_labels = list(df_agg["creneau"].cat.categories)

lignes_grille = []

#Cherche dans l'historique
for d in dates_annee:
    #Cherche tous les jours identiques, par exemple tous les 15 août entre 2019 et 2025
    proche = df_hist[
        (df_hist["date"].dt.month == d.month) &
        (df_hist["date"].dt.day == d.day)
    ]
    #Température moyenne historique par créneau, un profil de température moyen par jour et
    par créneau (quelle température observe-t-on habituellement à cette date et ce créneau )
    temp_climato = proche.groupby(
        "creneau",
        observed=True
    )["temp_moy"].mean()

    #Pour chaque créneau de chaque jour de 2026, on associe une température moyenne basée sur
    les années précédentes
    for cren in creneaux_labels:
        lignes_grille.append({
            "date": d,
```

```

        "mois": d.month,
        "creneau": cren,
        "temp_moy_base": temp_climato.get(cren, df_hist["temp_moy"].mean()),
        "saison": determiner_saison(d),
        "vacances": est_en_vacances(d),
        "jour_semaine": d.day_name(),
    })

```

```
grille = pd.DataFrame(lignes_grille)
```

```

print(f"Grille construite : {len(grille)} lignes ({len(dates_annee)} jours x
{len(creneaux_labels)} créneaux)")

```

Résultat : une grille de 1 460 lignes (365 jours x 4 créneaux)

On effectue à nouveau la prédiction sur TOUTE l'année 2026 à partir de la grille, en ajoutant à chaque fois un peu plus de chaleur par rapport à la normale : +0, +0,5, +1, +1,5, +2, +2,5, +3 °C. L'idée est de voir comment le nombre d'interventions prédit évolue si les températures augmentent.

```

deltas = [round(x * 0.5, 1) for x in range(1, 7)] # 0°C température historique normale, +
3°C = +3°C tout le temps
resultats_par_delta = {}

```

```

# Référence, servira de base de comparaison pour tous les chiffres clés
# la température de référence utilisée est la température moyenne historique du jour et
du créneau
# calcul de la prédiction sans augmentation de la température (Combien d'interventions
prévoir en 2026 si la météo ressemble à la moyenne historique ?)
g0 = grille.copy()
g0["temp_moy"] = g0["temp_moy_base"]
g0["prediction"] = modele_rf.predict(g0[var_predictives])
resultats_par_delta[0.0] = g0

```

```

# Prédiction du nombre d'interventions si une hausse uniforme de +0,5, 1°C etc est
observée (Combien d'interventions prévoir si toutes les températures augmentent de
+0,5/+1°C/...)

```

```

for delta in deltas:
    g = grille.copy()
    g["temp_moy"] = g["temp_moy_base"] + delta
    g["prediction"] = modele_rf.predict(g[var_predictives])
    resultats_par_delta[delta] = g

```

Pour calculer des chiffres clés sur l'impact du réchauffement climatique sur l'activité opérationnelle, nous mobilisons une fonction qui indique le total d'interventions par créneau (4), ce que cela représente sur 2 créneaux (6-18h et 18-6h) et ce que cela représente sur la journée entière (*in fine* sert à donner un résultat en fonction d'une date choisie par l'utilisateur et d'une température choisie). Nous calculons également le total du mois et de l'année en fonction d'une température. Ces chiffres clés sont mis en comparaison avec la prédiction de base, se basant sur un réchauffement climatique suivant les tendances des moyennes climatiques historiques.

```

# Création de créneaux type "garde"
creneaux_jour = ["Matin (6-12h)", "Après-midi (12-18h)"]

```

```

creneaux_nuit = ["Soir (18-24h)", "Nuit (0-6h)"]

#liste des scénarios de températures
deltas_export = [0.0] + deltas

#CALCUL DE LA MOYENNE HISTORIQUE
#Cherche tous les jours identiques, par exemple tous les 15 aout entre 2019 et 2025 (même
fonctionnement que la variable "proche" citée plus haut)
data_jour = {}
for d in dates_annee:
    d_str = d.strftime("%Y-%m-%d")

    proche = df_hist[
        (df_hist["date"].dt.month == d.month) &
        (df_hist["date"].dt.day == d.day)
    ]
    #Pour chaque jour, on calcule le nombre moyen d'interventions par créneau historique
    entre 2019 et 2025 (référence historique journalière)
    hist_par_creneau = proche.groupby("creneau", observed=True)
    ["nb_interventions"].mean()
    hist_ref = [int(round(float(hist_par_creneau.get(c, 0)))) for c in creneaux_labels]

    #Intégration des prédictions du modèle pour chaque jour/créneau et pour chaque scénario
    (calculées dans la cellule précédente)
    pred_par_delta = {}
    for delta in deltas_export:
        g = resultats_par_delta[delta]
        vals = [
            int(round(float(g[(g["date"] == d) & (g["creneau"] == c)]
["prediction"].sum()))))
            for c in creneaux_labels
        ]
        pred_par_delta[str(delta)] = vals
    #Stockage des données du jour pour export JSON
    data_jour[d_str] = {
        "jour_semaine": d.day_name(),
        "hist_ref": hist_ref,
        "pred_par_delta": pred_par_delta,
    }

    #Référence historique moyenne 2019-2025 (chiffres clés)
    #Nombre moyen d'interventions par mois
    hist_par_mois = {
        str(m): int(round(
            df_hist[df_hist["date"].dt.month == m]["nb_interventions"].sum()
            / df_hist["date"].dt.year.nunique()
        ))
        for m in range(1,13)
    }

    #Nombre moyen d'interventions par année (somme des mois)
    hist_annee = sum(hist_par_mois.values())

    #Regroupement des indicateurs historiques utilisés dans le dashboard
    hist_reference = {
        "annee": hist_annee,
        "mois_moyen": int(round(hist_annee / 12)),
        "semaine_moyenne": int(round(hist_annee / 52)),
    }

```

```

"creneau_jour": int(round(
    df_hist[df_hist["creneau"].isin(creneaux_jour)][ "nb_interventions"].sum()
    / df_hist["date"].dt.year.nunique()
)),

#Répartition historique jour/nuit
"creneau_nuit": int(round(
    df_hist[df_hist["creneau"].isin(creneaux_nuit)][ "nb_interventions"].sum()
    / df_hist["date"].dt.year.nunique()
)),
"par_mois": hist_par_mois
}

#Préparation des données pour l'export en JSON
data_globale = {
    "historique": hist_reference
}

for delta in deltas_export:
    g = resultats_par_delta[delta]
    total_annee = float(g["prediction"].sum())
    data_globale[str(delta)] = {
        "annee": int(round(total_annee)),
        "mois_moyen": int(round(total_annee / 12)),
        "semaine_moyenne": int(round(total_annee / 52)),

        "creneau_jour": int(round(float(g[g["creneau"].isin(creneaux_jour)]
["prediction"].sum()))),
        "creneau_nuit": int(round(float(g[g["creneau"].isin(creneaux_nuit)]
["prediction"].sum()))),
        "par_creneau": {
            c: int(round(float(g[g["creneau"] == c]["prediction"].sum())))
            for c in creneaux_labels
        },

        "par_mois": {
            str(m): int(round(float(g[g["mois"] == m]["prediction"].sum())))
            for m in range(1, 13)
        },
    }

#Clarifications des variables exportées
export = {
    "creneaux_labels": creneaux_labels,
    "deltas": deltas_export,
    "annee_reference": annee_cible,
    "data_jour": data_jour,
    "data_globale": data_globale,
}

#Ecriture du fichier json
chemin_json = Path(r"emplacement du dossier qui contiendra les .js, .css et .html")
#NB : Bien penser à appeler le fichier "data.json"

#ÉTAPE FINALE : on exporte les résultats calculés dans un fichier .json #afin que le
tableau de bord puisse simplement lire le fichier et qu'il n'ait pas #à refaire tous les
calculs à chaque interaction avec les choix utilisateurs

```

```
#Les jeux de données créés répondent à un besoin différencié du dashboard #final, un
onglet est plutôt réservé à une utilisation journalière (que va-t-il se passer
aujourd'hui selon le modèle prédictif) et une utilisation #plutôt destinée à du recueil
d'informations synthétiques #mensuelles/annuelles.
```

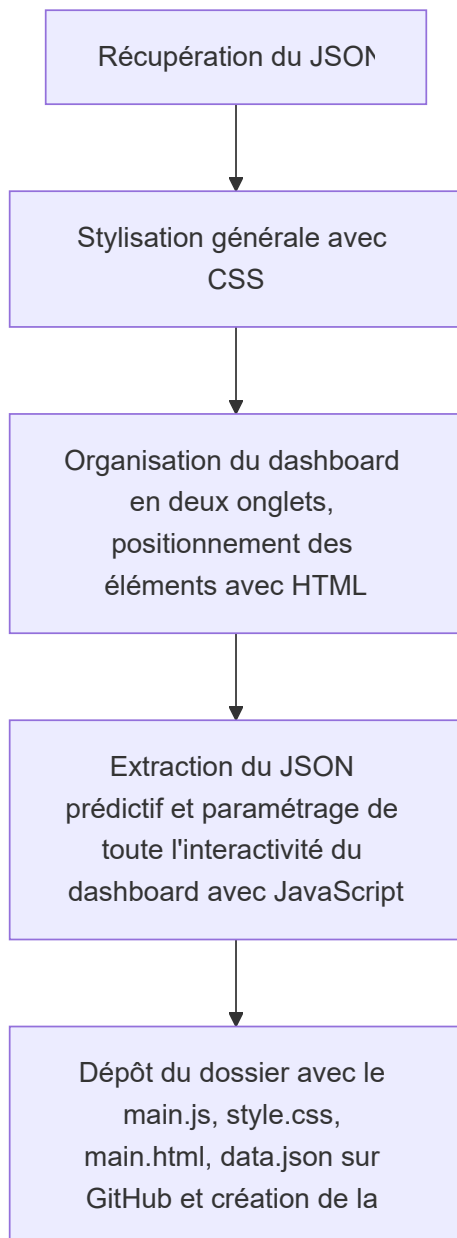
```
with open(chemin_json, "w", encoding="utf-8") as f:
    json.dump(export, f, ensure_ascii=False)
```

```
#vérification du fichier exporté pour s'assurer de la passation sur Github
taille_ko = os.path.getsize(chemin_json) / 1024
print(f"Export terminé : {chemin_json.name} ({taille_ko:.0f} Ko)")
```

Au final, sera publié sur GitHub uniquement ce fichier agrégé et non pas le fichier brut des interventions.

3ème étape : la création du dashboard en lui-même et sa publication

La création du dashboard suit la logique suivante :



Les fichiers .css, .js et .html sont disponibles sur le dépôt GitHub et commentés.